

Password Hash Cracking Using John the Ripper

4.1 Objective

The objective of this exercise was to demonstrate how password hashes can be subjected to dictionary-based password cracking using **John the Ripper**.

The exercise involved:

- Creating a controlled password hash.
- Preparing a small password wordlist.
- Using John the Ripper to test candidate passwords.
- Recovering the original password from the hash.
- Verifying the result using John's `--show` option.

The exercise was performed entirely within the authorized Kali Linux laboratory environment.

4.2 Hash Preparation

A known test password was selected for the controlled demonstration:

```
password123
```

The SHA-256 hash of the password was generated and stored in a file named:

```
~/hashes.txt
```

The hash was used as the input for John the Ripper.

A separate wordlist was created containing several candidate passwords:

```
123456  
password  
admin  
welcome  
password123  
qwerty
```

The wordlist was saved as:

```
~/john-wordlist.txt
```

The wordlist contained both incorrect candidates and the correct password, allowing the cracking process to be demonstrated in a controlled manner.

REPORT BY MANASVI RAJPUT

4.3 John the Ripper Execution

John the Ripper was executed using the SHA-256 format and the prepared wordlist.

The following command was used:

```
john --format=raw-sha256 --wordlist=~/.john-wordlist.txt ~/.hashes.txt
```

John identified the input as a SHA-256 password hash and loaded one hash for cracking.

The tool tested the candidate passwords from the supplied wordlist.

The correct password was identified as:

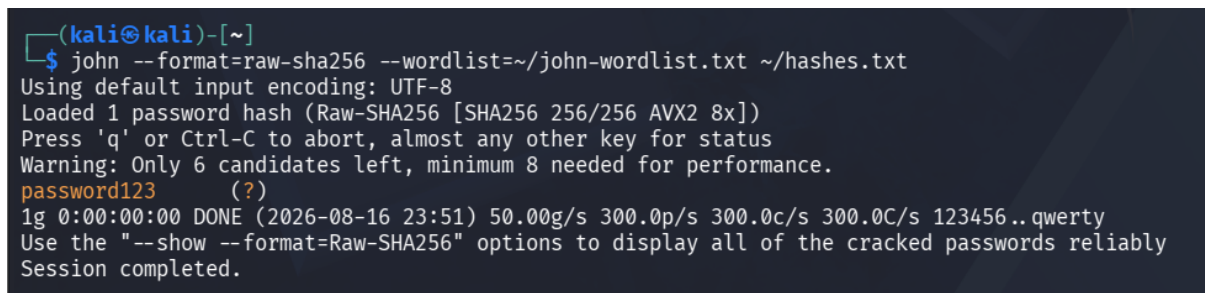
```
password123
```

The output indicated:

```
1g ... DONE
```

and:

```
Session completed.
```



```
(kali@kali)-[~]  
$ john --format=raw-sha256 --wordlist=~/.john-wordlist.txt ~/.hashes.txt  
Using default input encoding: UTF-8  
Loaded 1 password hash (Raw-SHA256 [SHA256 256/256 AVX2 8x])  
Press 'q' or Ctrl-C to abort, almost any other key for status  
Warning: Only 6 candidates left, minimum 8 needed for performance.  
password123 (?)  
1g 0:00:00:00 DONE (2026-08-16 23:51) 50.00g/s 300.0p/s 300.0c/s 300.0C/s 123456..qwerty  
Use the "--show --format=Raw-SHA256" options to display all of the cracked passwords reliably  
Session completed.
```

Figure 1: John the Ripper successfully cracking the SHA-256 password hash

4.4 Verification of Cracked Password

After the cracking process completed, John's --show option was used to display the recovered password.

The following command was executed:

```
john --show --format=raw-sha256 ~/.hashes.txt
```

The output showed:

```
?:password123
```

followed by:

```
1 password hash cracked, 0 left
```

This confirmed that the single password hash had been successfully recovered.

```
(kali@kali)-[~]  
$ john --show --format=raw-sha256 ~/hashes.txt  
?:password123  
  
1 password hash cracked, 0 left
```

Figure 2: Verification of the password recovered by John the Ripper

4.5 Result

The John the Ripper exercise successfully recovered the password corresponding to the supplied SHA-256 hash.

Summary

Parameter	Result
Hash algorithm	SHA-256
John format	raw-sha256
Number of hashes	1
Wordlist	john-wordlist.txt
Password recovered	password123
Hashes cracked	1
Hashes remaining	0
Result	Successful

The final John output confirmed:

```
1 password hash cracked, 0 left
```

4.6 Security Implications

Password hashes are designed to avoid storing passwords directly. However, if an attacker obtains password hashes, they may attempt to recover the original passwords by testing candidate values against the hashes.

The demonstration shows that passwords that are common, predictable, or present in password dictionaries can be recovered relatively easily when an attacker has access to the corresponding hash.

The use of a strong password is therefore important even when passwords are stored in hashed form.

4.7 Security Recommendations

The following measures can reduce the risk associated with password-hash cracking:

1. Use Strong Passwords

Passwords should be long, unique, and difficult to predict.

2. Avoid Common Passwords

Passwords such as:

```
password  
123456  
qwerty  
admin
```

should not be used.

3. Use Appropriate Password Hashing Algorithms

Applications should use password-specific hashing algorithms designed to be computationally expensive, such as Argon2id, bcrypt, or scrypt, rather than relying on fast general-purpose hashes for password storage.

4. Use Unique Passwords

A different password should be used for every account and service to reduce the impact of credential compromise.

5. Protect Password Hashes

Password databases should be protected using appropriate file permissions and access controls.

6. Use Multi-Factor Authentication

MFA provides an additional security layer even if a password is compromised.

7. Monitor for Credential Compromise

Organizations should monitor authentication systems and investigate suspicious login activity.

4.8 Limitations

This exercise was performed using a **controlled test hash** generated specifically for the laboratory demonstration.

The exercise demonstrates the password-cracking technique but does not represent the compromise of a real user's password or an unauthorized system.

The wordlist was intentionally small so that the demonstration could be completed quickly and the result could be verified.

4.9 Conclusion

The John the Ripper exercise successfully demonstrated password-hash cracking using a dictionary-based approach.

A SHA-256 password hash was supplied to John together with a controlled wordlist. John successfully identified the original password, `password123`, and the result was independently confirmed using the `--show` option.

The final output:

```
1 password hash cracked, 0 left
```

confirmed the successful completion of the exercise.

This demonstration highlights the importance of using strong, unique passwords, secure password-specific hashing algorithms, proper protection of password databases, and additional authentication mechanisms such as MFA.